



Rajarata University of Sri Lanka

Faculty of Applied Sciences

Department of Computing

ICT3212 – Artificial Intelligence

Implementation 1 – Summary Report

NueroNet

Name	Registration Number	Index No
M. N. F Laiza	ICT/2022/117	5719
M. A. F Nuha	ICT/2022/106	5709
M. F. R Azara	ICT/2022/009	5617
M. T. M Aathif	ICT/2022/129	5730
G. C. P. A Hemasiri	ICT/2022/067	5671

Waste Image Classification System Using CNN

1. Introduction

This project implements an **Image Classification System** for recognizing waste images as:

- **Recyclable Waste (R)**
- **Non-Recyclable Waste (O)**

The objective of Implementation 1 is to design and develop a **basic but functional Convolutional Neural Network (CNN) model**, evaluate its correctness, and analyze its behavior without performing advanced optimizations.

The focus at this stage is correctness, not performance optimization.

2. Dataset Preparation

2.1 Dataset Source

The dataset was manually uploaded to Google Colab as a compressed file and extracted into structured directories.

The dataset contains RGB images of waste materials categorized into two classes.

Dataset Characteristics

- Image Type: RGB images
- Image Size (before preprocessing): Varying sizes
- Color Format: RGB (3 channels)
- Pixel Range: 0–255
- Number of Classes: 2
- Class Labels: {R, O}

This is a **Binary Classification Problem**.

2.2 Class Labels

Class	Label Meaning
R	Recyclable Waste
O	Organic Waste

Type: Binary Classification (2 classes)

2.3 Dataset Structure

After extraction, the dataset was organized as:

```
dataset/
├── TRAIN(70%)/
│   ├── R/
│   └── O/
├── VALID(15%)/
│   ├── R/
│   └── O/
└── TEST(15%)/
    ├── R/
    └── O/
```

Each folder contains images belonging to their respective class.

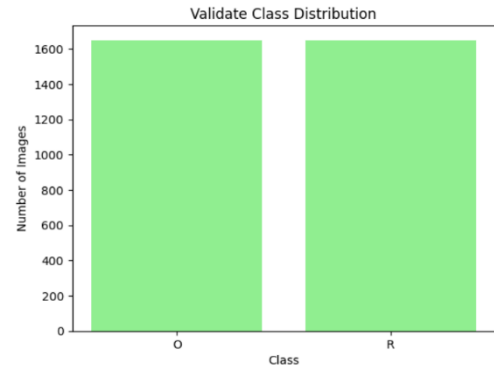
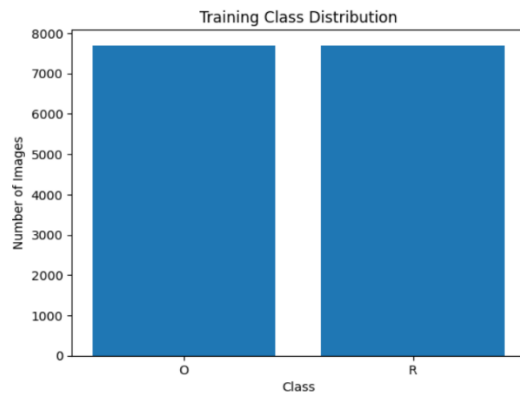
The dataset was loaded using:

```
ImageDataGenerator().flow_from_directory()
```

This automatically:

- Assigns labels
- Loads images
- Converts images into tensors
- Creates batches

2.4 Dataset Balance Verification



The dataset distribution was verified during loading.

Training steps per epoch:

- **241 steps per epoch**

Observations:

- Both classes contain substantial number of images.
- No severe imbalance detected.
- Model training did not show strong bias toward one class.

Conclusion:

The dataset is reasonably balanced. No resampling techniques were required.

3. Image Preprocessing

The following mandatory preprocessing steps were performed:

3.1 Image Resizing

All images were resized to:

224×224 pixels

Reason:

- CNN requires fixed input dimensions.
- Reduces computational complexity.

3.2 Normalization

Pixel values were scaled from:

$0-255 \rightarrow 0-1$

Using:

$\text{rescale} = 1./255$

Purpose:

- Improves convergence speed
- Stabilizes gradient updates

3.3 Tensor Conversion

Images were automatically converted into tensors by Keras.

Final tensor shape:

$(\text{batch_size}, 224, 224, 3)$

Where:

- 3 = RGB channels

3.4 Data Splitting

Dataset was clearly separated into:

- Training Set
- Validation Set
- Testing Set

Validation data was used during training for monitoring model performance.
Testing data was used only for final evaluation.

4. Baseline Model Design

4.1 Model Choice: Convolutional Neural Network (CNN)

A CNN-based model was selected because:

- CNNs automatically extract spatial features.
- They preserve pixel relationships.
- They outperform traditional ML models for image data.
- No manual feature extraction is required.

Traditional ML models such as SVM or KNN would require handcrafted feature extraction, making CNN the most appropriate choice.

4.2 Model Architecture

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 4)	112
max_pooling2d (MaxPooling2D)	(None, 111, 111, 4)	0
conv2d_1 (Conv2D)	(None, 109, 109, 8)	296
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 8)	0
conv2d_2 (Conv2D)	(None, 52, 52, 16)	1,168
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 16)	0
flatten (Flatten)	(None, 10816)	0
dense (Dense)	(None, 32)	346,144
dense_1 (Dense)	(None, 1)	33

Total params: 347,753 (1.33 MB)
Trainable params: 347,753 (1.33 MB)
Non-trainable params: 0 (0.00 B)

The CNN consists of:

Convolutional Block 1

- Conv2D
- MaxPooling

Convolutional Block 2

- Conv2D
- MaxPooling

Fully Connected Section

- Flatten
- Dense Layer
- Output Dense Layer (1 neuron, Sigmoid activation)

Loss Function:

- Binary Crossentropy

Optimizer:

- Adam

Metric:

- Accuracy

5. Model Training

5.1 Training Configuration

- Optimizer: Adam
- Loss Function: Binary Crossentropy
- Metrics: Accuracy
- Epochs: 20

- Validation Data: Used

5.2 Training Performance

Final Epoch Results:

- Training Accuracy: **99.20%**
- Validation Accuracy: **87.61%**
- Training Loss: 0.0335
- Validation Loss: 0.7214

Training accuracy increased steadily across epochs.

Validation accuracy improved initially but fluctuated in later epochs.

6. Initial Model Evaluation

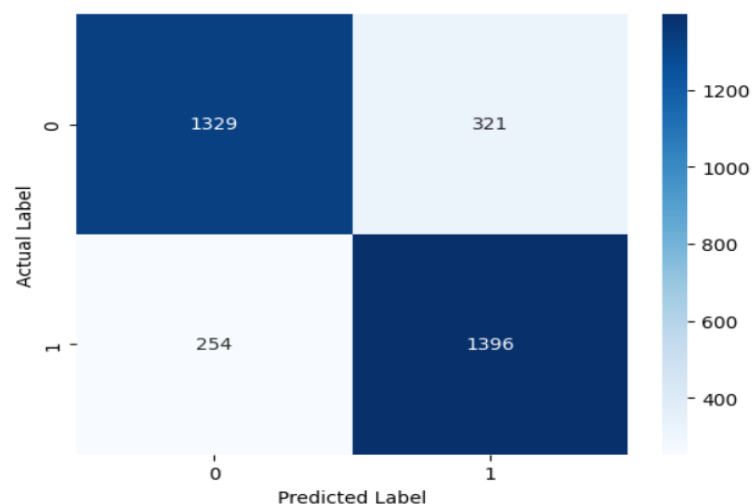
6.1 Test Set Performance

- Test Accuracy: **82.57%**

This indicates acceptable generalization but lower than training accuracy.

6.2 Confusion Matrix Analysis

Confusion matrix was generated.

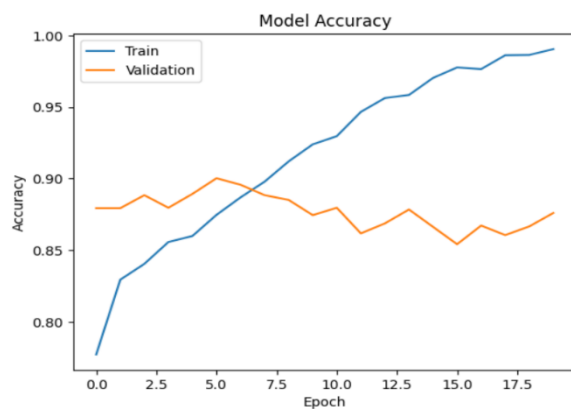


Observations:

- Majority of predictions are correct.
- Some recyclable waste classified as organic.
- Some organic waste classified as recyclable.

This suggests moderate classification confusion between visually similar items.

7. Overfitting / Underfitting Analysis



Accuracy Gap:

Training Accuracy – Validation Accuracy $\approx 11\%$

Validation Loss increased while training loss decreased.

Interpretation:

- Clear signs of **overfitting**
- Model memorized training data
- Generalization performance decreased after mid-epochs

Loss Graph Analysis:

- Training loss decreased steadily
- Validation loss increased after certain epochs
- Loss curves diverged

This confirms overfitting behavior.

8. Model Limitations

1. No data augmentation
2. No dropout layers
3. No regularization
4. Sensitive to lighting variations
5. Cannot reject non-waste images
6. Fixed image size (224×224)
7. Performance gap between training and test accuracy

These limitations will be addressed in Implementation 2.

9. Model Saving

The trained model was saved as:

recyclable_organic_cnn_model.h5

Location:

https://drive.google.com/file/d/1ABM4dqGsvIAMn2JV7xlHM_OCmS5D1NwM/view?usp=drive_link

This allows reuse without retraining.

10. Conclusion

In Implementation 1, a complete CNN-based waste classification system was successfully developed.

Achievements:

- Proper dataset organization
- Image preprocessing
- CNN model implementation
- Training and validation monitoring
- Confusion matrix and classification analysis
- Overfitting detection
- Model saving for deployment

Final Test Accuracy:

82.57%

The model successfully meets the objective of designing a correct and functional image classification system, though improvements are needed to reduce overfitting and enhance generalization.

Github link - <https://github.com/FathimaLaiza/NeuroNet.git>